

Mô phỏng & Cài đặt (Simulation)

Nội dung

1. Lập trình thi đấu là gì?
 - Cách hoạt động của Online Judge
 - Các kết quả phổ biến
2. Mô phỏng & Cài đặt là gì?
3. Quy trình giải một bài CP
 - Bước 1: Đọc đề cẩn thận
 - Bước 2: Chạy tay ví dụ
4. Độ phức tạp thuật toán (Big O) — Thuật toán của bạn nhanh hay chậm?
 - Big O là gì?
 - Các bậc phổ biến (từ nhanh đến chậm)
 - Cách tính Big O của code
 - Quy tắc 10^8 — Thuật toán cần nhanh cỡ nào?
5. Template code C++
 - Ghi chú cho Python
 - Kiểu dữ liệu: int vs long long
6. Ví dụ 1: Bài "Weird Algorithm" (Thuật toán lạ)
 - Chạy tay với $n = 3$
 - Phân tích
 - Code
7. Ví dụ 2: Bài "Missing Number" (Số bị thiếu)
 - Chạy tay
 - Phân tích
 - Code
8. Ví dụ 3: Bài "Repetitions" (Chuỗi lặp dài nhất)
 - Chạy tay
 - Phân tích
 - Code
9. Các lỗi phổ biến cần tránh
 - Lỗi 1: Tràn số (Integer Overflow)

- Lỗi 2: Sai chỉ số (Off-by-one Error)
- Lỗi 3: Không đọc kỹ đề
- Lỗi 4: Quên xử lý edge case
- Lỗi 5: Dùng endl thay vì "\n"
- Lỗi 6: Input chậm trong Python

10. Mẹo khi nộp bài

- Tóm tắt bài học

Chào mừng bạn đến với bài học đầu tiên của khóa Lập trình thi đấu — Đồng! Nếu đây là lần đầu bạn tiếp xúc với lập trình thi đấu (competitive programming — viết tắt là CP), thì đây chính là nơi bắt đầu. Hãy yên tâm, chúng ta sẽ đi từng bước một.

1. Lập trình thi đấu là gì?

Lập trình thi đấu (Competitive Programming) là hình thức giải các bài toán thuật toán bằng cách viết chương trình, rồi nộp lên một hệ thống chấm tự động (online judge). Hệ thống sẽ chạy chương trình của bạn với nhiều bộ test, và cho biết kết quả: đúng (AC), sai (WA), quá thời gian (TLE), lỗi chạy (RE),...

Cách hoạt động của Online Judge

Bạn viết code → Nộp bài → Hệ thống biên dịch → Chạy với nhiều test → Trả kết quả

Mỗi bài toán có:

- **Đề bài:** Mô tả bài toán, giới hạn dữ liệu, ví dụ input/output.
- **Giới hạn thời gian (Time Limit):** Thường 1-2 giây. Chương trình phải chạy xong trong thời gian này.
- **Giới hạn bộ nhớ (Memory Limit):** Thường 256MB.
- **Nhiều test ẩn:** Bạn chỉ thấy 1-2 ví dụ, nhưng hệ thống có 10-20 test (hoặc hơn) để kiểm tra.

Các kết quả phổ biến

KẾT QUẢ	Ý NGHĨA
AC (Accepted)	Đúng tất cả test — chúc mừng!
WA (Wrong Answer)	Sai đáp án ở ít nhất 1 test
TLE (Time Limit Exceeded)	Chạy quá thời gian — thuật toán chậm
RE (Runtime Error)	Lỗi khi chạy (truy cập ngoài mảng, chia cho 0,...)
MLE (Memory Limit Exceeded)	Dùng quá nhiều bộ nhớ

2. Mô phỏng & Cài đặt là gì?

Dạng bài Simulation / Implementation là dạng bài mà đề bài mô tả rõ ràng quy trình cần làm, và nhiệm vụ của bạn là chuyển mô tả thành code một cách chính xác. Không cần thuật toán phức tạp — chỉ cần đọc hiểu đề và cài đặt cẩn thận.

Tại sao đây là bài học đầu tiên? Vì trước khi học các thuật toán nâng cao, bạn cần rèn luyện khả năng:

- Đọc hiểu đề bài (đặc biệt là phần giới hạn và ví dụ)
- Viết code sạch, đúng, không bug
- Test và debug hiệu quả

3. Quy trình giải một bài CP

Dù là bài dễ hay khó, quy trình luôn gồm 6 bước:

Bước 1: Đọc đề (ít nhất 2 lần!)

Bước 2: Phân tích — chạy tay ví dụ

Bước 3: Thiết kế thuật toán — ước lượng độ phức tạp

Bước 4: Viết code

Bước 5: Test với ví dụ trong đề + tự nghĩ thêm test

Bước 6: Kiểm tra edge case → Nộp bài

Bước 1: Đọc đề cẩn thận

Đọc ít nhất 2 lần. Lần đầu để hiểu tổng quát, lần sau để chú ý chi tiết. Đặc biệt chú ý:

- **Giới hạn dữ liệu (n, m , giá trị tối đa):** Quyết định thuật toán nào được phép.
- **Định dạng input/output:** Input có bao nhiêu dòng? Các số cách nhau bởi dấu cách hay xuống dòng? Output có cần xuống dòng cuối không?
- **Các ví dụ:** Tự chạy tay trước khi viết code. Nếu bạn không hiểu tại sao ví dụ cho kết quả đó, thì bạn chưa hiểu đề!

Bước 2: Chạy tay ví dụ

Lấy giấy bút (hoặc text editor) và tự tay mô phỏng quá trình xử lý với ví dụ trong đề. Đây là bước nhiều người bỏ qua, nhưng lại cực kỳ quan trọng. Chạy tay giúp bạn:

- Xác nhận bạn đã hiểu đúng đề
- Phát hiện các trường hợp đặc biệt
- Hình dung cách viết code

4. Độ phức tạp thuật toán (Big O) — Thuật toán của bạn nhanh hay chậm?

Trong CP, viết code đúng chưa đủ — code còn phải đủ nhanh. Để đánh giá tốc độ, ta dùng khái niệm Big O.

Big O là gì?

Big O mô tả số phép tính mà thuật toán cần thực hiện, tính theo kích thước đầu vào n . Ta chỉ quan tâm đến phần tăng nhanh nhất (bỏ hằng số và bậc thấp hơn).

Ví dụ trực quan: Giả sử bạn cần tìm một tên trong danh sách n học sinh.

- **Duyệt từ đầu đến cuối:** Kiểm tra từng tên, tối đa n lần $\rightarrow O(n)$.
- **Nếu danh sách đã sắp xếp theo ABC:** Mở giữa sách, so sánh, bỏ nửa không cần $\rightarrow$ mỗi bước loại nửa danh sách $\rightarrow$ chỉ cần $\log_2 n$ lần $\rightarrow O(\log n)$.

Với $n = 1,000,000$:

- $O(n) = 1,000,000$ phép tính
- $O(\log n) = 20$ phép tính (vì $\log_2 10^6 \approx 20$)

Chênh lệch 50,000 lần! Đó là lý do Big O quan trọng.

Các bậc phổ biến (từ nhanh đến chậm)

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Hãy hình dung với $n = 10^6$:

BẬC	SỐ PHÉP TÍNH (ƯỚC LƯỢNG)	NHANH?
$O(1)$	1	Cực nhanh
$O(\log n)$	20	Cực nhanh
$O(n)$	10^6	Nhanh (< 1 giây)
$O(n \log n)$	2×10^7	Nhanh (< 1 giây)
$O(n^2)$	10^{12}	QUÁ CHẬM (hàng giờ)

Cách tính Big O của code

Quy tắc 1: Một vòng lặp đơn chạy n lần $\rightarrow O(n)$

C++

```
for (int i = 0; i < n; i++) { ... } // O(n)
```

Python

```
for i in range(n):  
    ... # O(n)
```

Quy tắc 2: Hai vòng lặp lồng nhau $\rightarrow O(n^2)$

C++

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++) { ... } //  $O(n^2)$ 
```

Python

```
for i in range(n):  
    for j in range(n):  
        ... #  $O(n^2)$ 
```

Quy tắc 3: Vòng lặp mà biến nhân đôi hoặc chia đôi $\rightarrow O(\log n)$

C++

```
for (int i = 1; i < n; i *= 2) { ... } //  $O(\log n)$ 
```

Python

```
i = 1  
while i < n:  
    ...  
    i *= 2 #  $O(\log n)$ 
```

Quy tắc 4: Nối tiếp nhau $\rightarrow$ lấy cái lớn nhất

C++

```
sort(a.begin(), a.end()); //  $O(n \log n)$   
for (int i = 0; i < n; i++) { ... } //  $O(n)$   
// Tổng:  $O(n \log n)$  (bỏ  $O(n)$  vì nhỏ hơn)
```

Python

```
a.sort() #  $O(n \log n)$   
for x in a: #  $O(n)$   
    ...  
# Tổng:  $O(n \log n)$ 
```

Quy tắc 10^8 — Thuật toán cần nhanh cỡ nào?

Máy tính thực hiện khoảng 10^8 phép tính mỗi giây (ước lượng an toàn). Thời gian giới hạn thường là 1-2 giây. Dựa vào giới hạn n trong đề bài, tra bảng sau:

GIỚI HẠN n	ĐỘ PHỨC TẠP CHẤP NHẬN ĐƯỢC	VÍ DỤ THUẬT TOÁN
$n \leq 10$	$O(n!), O(2^n)$	Duyệt hoán vị, duyệt tập con
$n \leq 20$	$O(2^n), O(n^2 \cdot 2^n)$	Bitmask, meet in the middle
$n \leq 500$	$O(n^3)$	Ba vòng lặp lồng nhau
$n \leq 5,000$	$O(n^2)$	Hai vòng lặp lồng nhau
$n \leq 10^6$	$O(n \log n)$	Sắp xếp, cây nhị phân
$n \leq 10^8$	$O(n)$	Duyệt 1 lần, prefix sum
$n \leq 10^{18}$	$O(\log n)$ hoặc $O(1)$	Binary search, công thức toán

Cách dùng bảng này: Nhìn giới hạn n trong đề $\rightarrow$ tra cột giữa $\rightarrow$ biết cần thuật toán nhanh cỡ nào.

Ví dụ: Đề cho $n \leq 10^5$. Tra bảng $\rightarrow$ cần $O(n \log n)$ hoặc tốt hơn. Thuật toán $O(n^2) = (10^5)^2 = 10^{10}$ phép tính $\rightarrow$ chạy khoảng 100 giây $\rightarrow$ TLE!

Mẹo cho người mới: Ở giai đoạn đầu, đừng lo quá nhiều về Big O. Cứ code cách đơn giản nhất trước. Nếu bị TLE, quay lại đọc giới hạn và nghĩ cách nhanh hơn. Dần dần bạn sẽ có trực giác!

Ghi chú cho Python:

- Python thường chậm hơn C++, nên ở cùng một giới hạn, bạn cần cẩn thận hơn với thuật toán.
- Với $n \leq 2 \times 10^5$ hay 10^6 , Python vẫn xử lý tốt nếu thuật toán là $O(n)$ hoặc $O(n \log n)$ và code gọn.
- Tránh các vòng lặp $O(n^2)$ lồng nhau khi n lớn.

5. Template code C++

Hầu hết các bạn CP dùng C++ vì tốc độ nhanh và thư viện chuẩn (STL) rất mạnh. Đây là template cơ bản:

C++

```
#include <bits/stdc++.h> // Include tất cả thư viện chuẩn
using namespace std; // Không cần viết std:: trước mỗi hàm

int main() {
    ios_base::sync_with_stdio(false); // Tăng tốc nhập/xuất
    cin.tie(NULL); // Bỏ đồng bộ cin/cout

    // File I/O (nếu đề yêu cầu)
    // freopen("input.txt", "r", stdin);
    // freopen("output.txt", "w", stdout);

    // Đọc input
    int n;
```

```

    cin >> n;

    // Xử lý
    // ... viết thuật toán ở đây ...

    // Xuất output
    // cout << ans << "\n";

    return 0;
}

```

Giải thích từng dòng:

DÒNG	Ý NGHĨA
#include <bits/stdc++.h>	Include toàn bộ thư viện C++ (chỉ dùng trong CP, không dùng trong thực tế)
using namespace std;	Cho phép viết cout thay vì std::cout
ios_base::sync_with_stdio(false);	Tắt đồng bộ C/C++ I/O → nhanh hơn 5-10 lần
cin.tie(NULL);	Bỏ flush cout trước mỗi cin → nhanh hơn
"\n" thay endl	endl flush buffer (chậm), "\n" không flush (nhanh)

Ghi chú cho Python

Python Template

```

import sys

def main():
    input = sys.stdin.readline
    # File I/O (nếu đề yêu cầu)
    # sys.stdin = open("input.txt", "r")
    # sys.stdout = open("output.txt", "w")

    # Đọc input
    n = int(input())

    # Xử lý
    # ... viết thuật toán ở đây ...

    # Xuất output
    # print(ans)

if __name__ == "__main__":
    main()

```

- `sys.stdin.readline`: Cách đọc input nhanh trong Python.
- Python không bị tràn số nguyên như C++ `int`; số nguyên của Python có thể lớn tùy ý.

- Tuy nhiên, Python chậm hơn C++, nên cần chú ý tối ưu input/output.
- Với output lớn, có thể gom kết quả vào list rồi dùng `"".join(...)` hoặc `"\n".join(...)`.

Kiểu dữ liệu: int vs long long

Kiểu	Phạm vi	Khi nào dùng
int	-2×10^9 đến 2×10^9	Giá trị $\leq 2 \times 10^9$
long long	-9×10^{18} đến 9×10^{18}	Giá trị hoặc tổng có thể $> 2 \times 10^9$

Quy tắc đơn giản: Nếu không chắc, dùng long long cho an toàn. Hoặc dùng quy tắc: nếu $n \times \text{max_value}$ có thể vượt 2×10^9 , dùng long long.

Ví dụ: Tính tổng n số, mỗi số đến 10^9 , n đến 10^5 . Tổng tối đa $= 10^5 \times 10^9 = 10^{14} \rightarrow$ cần long long!

Ghi chú Python: Trong Python, kiểu int đã hỗ trợ số nguyên lớn, nên bạn không cần lo long long. Vấn đề chính của Python là tốc độ, không phải tràn số.

6. Ví dụ 1: Bài "Weird Algorithm" (Thuật toán lạ)

Đề bài: Cho số nguyên dương n . Lặp lại quy trình sau cho đến khi $n = 1$:

- Nếu n chẵn: $n = n/2$
- Nếu n lẻ: $n = 3n + 1$

In ra tất cả các giá trị của n trong quá trình.

Giới hạn: $1 \leq n \leq 10^6$.

Chạy tay với $n = 3$

- $n = 3$ (lẻ) $\rightarrow n = 3 \times 3 + 1 = 10$
- $n = 10$ (chẵn) $\rightarrow n = 10/2 = 5$
- $n = 5$ (lẻ) $\rightarrow n = 3 \times 5 + 1 = 16$
- $n = 16$ (chẵn) $\rightarrow n = 16/2 = 8$
- $n = 8$ (chẵn) $\rightarrow n = 8/2 = 4$
- $n = 4$ (chẵn) $\rightarrow n = 4/2 = 2$
- $n = 2$ (chẵn) $\rightarrow n = 2/2 = 1$
- $\rightarrow$ Dừng!

Output: 3 10 5 16 8 4 2 1

Phân tích

- Đây là bài mô phỏng thuần túy: làm đúng theo mô tả, không cần sáng tạo thuật toán.
- **Bẫy:** Dù $n \leq 10^6$, trong quá trình tính, n có thể tăng rất lớn trước khi giảm về 1. Ví dụ: $n = 113383$ sẽ tăng lên hơn $2.5 \times 10^9 \rightarrow$ tràn int! Phải dùng long long.

Code

C++

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long n; // PHẢI dùng long long!
    cin >> n;
    while (true) {
        cout << n;
        if (n == 1) break;
        cout << " ";
        if (n % 2 == 0) n /= 2;
        else n = 3 * n + 1;
    }
    cout << "\n";
}
```

Python

```
n = int(input())
ans = []
while True:
    ans.append(str(n))
    if n == 1:
        break
    if n % 2 == 0:
        n //= 2
    else:
        n = 3 * n + 1
print(" ".join(ans))
```

Ghi chú Python:

- Dùng // để chia nguyên.
- Thường nên lưu kết quả vào list **ans**, rồi cuối cùng `print(" ".join(ans))` để output gọn và nhanh.

7. Ví dụ 2: Bài "Missing Number" (Số bị thiếu)

Đề bài: Cho $n - 1$ số phân biệt, mỗi số nằm trong đoạn $[1, n]$. Tìm số bị thiếu.

Giới hạn: $2 \leq n \leq 2 \times 10^5$.

Chạy tay

Input: $n = 5$, các số: 2 3 1 5

- Tổng $1 + 2 + 3 + 4 + 5 = 15$
- Tổng các số cho: $2 + 3 + 1 + 5 = 11$
- Số thiếu $= 15 - 11 = 4$

Phân tích

Tổng các số từ 1 đến n là $\frac{n(n+1)}{2}$. Trừ đi tổng các số đã cho, ta được số bị thiếu.

Độ phức tạp: $O(n)$ — duyệt 1 lần để tính tổng.

Code

C++

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    long long total = (long long)n * (n + 1) / 2;
    long long sum = 0;
    for (int i = 0; i < n - 1; i++) {
        int x;
        cin >> x;
        sum += x;
    }
    cout << total - sum << "\n";
}
```

Python

```
n = int(input())
arr = list(map(int, input().split()))
total = n * (n + 1) // 2
s = sum(arr)
print(total - s)
```

Lưu ý: $(\text{long long})n * (n + 1) / 2$ — ép kiểu long long trước khi nhân để tránh tràn số. Nếu $n = 2 \times 10^5$ thì $n \times (n + 1) \approx 4 \times 10^{10} \rightarrow$ tràn int!

Ghi chú Python:

- Python không lo tràn số ở phép tính này.
- Có thể viết cực ngắn:

```
n = int(input())
print(n * (n + 1) // 2 - sum(map(int, input().split())))
```

Tuy nhiên với người mới, nên viết tách biến `total`, `s` cho dễ đọc.

8. Ví dụ 3: Bài "Repetitions" (Chuỗi lặp dài nhất)

Đề bài: Cho một chuỗi DNA gồm các ký tự A, C, G, T. Tìm độ dài của đoạn liên tiếp dài nhất gồm cùng một ký tự.

Chạy tay

Input: ATTCGGGA

```
A T T C G G G A
^
  ^ ^
    ^
      ^ ^ ^
        ^ -> A: độ dài 1
          ^ ^ ^ -> T: độ dài 2
            ^ -> C: độ dài 1
              ^ ^ ^ -> G: độ dài 3
                ^ -> A: độ dài 1
```

Đoạn dài nhất: GGG, độ dài = 3. Output: 3.

Phân tích

Duyệt từ trái sang phải, duy trì biến đếm `cnt`:

- Nếu ký tự hiện tại giống ký tự trước $\rightarrow$ tăng `cnt`
- Nếu khác $\rightarrow$ reset `cnt = 1`
- Luôn cập nhật `ans = max(ans, cnt)`

Code

C++

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s;
    cin >> s;
    int ans = 1, cnt = 1;
    for (int i = 1; i < (int)s.size(); i++) {
        if (s[i] == s[i - 1]) cnt++;
        else cnt = 1;
    }
    cout << ans << endl;
}
```

```

        ans = max(ans, cnt);
    }
    cout << ans << "\n";
}

```

Python

```

s = input().strip()
ans = 1
cnt = 1
for i in range(1, len(s)):
    if s[i] == s[i - 1]:
        cnt += 1
    else:
        cnt = 1
    ans = max(ans, cnt)
print(ans)

```

Ghi chú Python: `input().strip()` giúp bỏ ký tự xuống dòng/thừa khoảng trắng ở hai đầu.

9. Các lỗi phổ biến cần tránh

Dưới đây là những lỗi mà rất nhiều người mới mắc phải. Hãy đọc kỹ để tránh!

Lỗi 1: Tràn số (Integer Overflow)

C++

```

// SAI - tràn số khi n lớn!
int n = 100000;
int total = n * (n + 1) / 2; // n * (n+1) = 10^10 -> tràn int!
// ĐÚNG
long long total = (long long)n * (n + 1) / 2;

```

Python

```

# Python không bị tràn số nguyên kiểu này
n = 100000
total = n * (n + 1) // 2

```

Ghi chú: Python tránh được lỗi tràn số nguyên thông thường, nhưng điều đó không có nghĩa là mọi code Python đều đủ nhanh.

Lỗi 2: Sai chỉ số (Off-by-one Error)

C++

```
// Mảng n phần tử, chỉ số từ 0 đến n-1
int a[100];
for (int i = 0; i <= n; i++) // SAI! i <= n truy cập a[n] - ngoài mảng
for (int i = 0; i < n; i++) // ĐÚNG
```

Python

```
a = [0] * n
for i in range(n + 1): # SAI nếu truy cập a[i] khi i = n
    ...
for i in range(n):      # ĐÚNG: i chạy từ 0 đến n-1
    ...
```

Lỗi 3: Không đọc kỹ đề

- Đề nói "in mỗi số trên một dòng" → dùng "\n", không phải " ".
- Đề nói "in trên cùng một dòng, cách nhau bởi dấu cách" → nhớ không có dấu cách thừa ở cuối.
- Đề nói "1-indexed" (đánh số từ 1) hay "0-indexed" (đánh số từ 0)?

Lỗi 4: Quên xử lý edge case

- $n = 0$ hoặc $n = 1$ — mảng rỗng hoặc chỉ có 1 phần tử.
- Tất cả phần tử giống nhau.
- Giá trị lớn nhất hoặc nhỏ nhất theo giới hạn.

Lỗi 5: Dùng endl thay vì "\n"

Trong các bài có output lớn, endl flush buffer mỗi lần → chậm gấp nhiều lần. Khi nộp bài, thường nên dùng "\n" thay vì endl.

Lưu ý: endl vẫn hữu ích khi debug vì nó ép in dữ liệu ra ngay lập tức (flush buffer). Điều này giúp bạn thấy output ngay tại thời điểm chương trình đang chạy. Nhưng trong lời giải chính thức để nộp, hãy ưu tiên "\n" để tránh chậm không cần thiết.

Lỗi 6: Input chậm trong Python

```
import sys
input = sys.stdin.readline
```

Khi dữ liệu lớn, cách này thường nhanh hơn input() thông thường.

10. Mẹo khi nộp bài

1. Test với ví dụ trong đề trước — nếu sai ví dụ, chắc chắn sai test ẩn.
2. Tự nghĩ thêm test — test nhỏ nhất ($n = 1$), test lớn nhất (giới hạn max), test đặc biệt.
3. Đọc lại code một lần trước khi nộp — nhiều khi phát hiện bug ngay lập tức.
4. Nếu WA: thử test nhỏ, in ra giá trị trung gian để debug.
5. Nếu TLE: kiểm tra độ phức tạp, có thể cần thuật toán tốt hơn.
6. Nếu RE: kiểm tra truy cập ngoài mảng, chia cho 0, stack overflow (đệ quy quá sâu).

Mẹo thêm cho Python:

- Với output rất lớn, thay vì print trong vòng lặp nhiều lần, hãy gom vào list rồi in một lần.
- Tránh dùng đệ quy sâu nếu không cần.
- Ưu tiên built-in như `sum`, `max`, `sort` vì chúng thường nhanh hơn tự viết vòng lặp Python thuần.

Tóm tắt bài học

KHÁI NIỆM	NỘI DUNG CHÍNH
CP là gì	Giải bài thuật toán, nộp lên online judge
Simulation	Đọc đề $\rightarrow$ chuyển mô tả thành code
Đánh giá độ phức tạp	Dựa vào n để chọn thuật toán phù hợp
Template C++ / Python	<code>bits/stdc++.h</code> , <code>ios_base</code> , <code>sys.stdin.readline</code>
Lỗi phổ biến	Tràn số, off-by-one, không đọc kỹ đề, input chậm